

JavaScript: this, прототипи, ООП, класи

Головна ідея: `this` визначається тим, **як** викликали функцію, а не де її оголосили (окрім стрілочних функцій). Прототипи — механізм, через який об'єкти "успадковують" властивості одне від одного. Класи — сучасний, зручний синтаксис поперх того самого прототипного механізму, що дозволяє структурувати код у стилі ООП: інкапсуляція, наслідування, публічний/приватний інтерфейс.

1. Контекст виконання: this

```
const user = {
  username: "Victor",
  showName() {
    console.log(this.username); // this -- об'єкт, у контексті якого викликали метод
  },
};
user.showName(); // "Victor"
```

Алгоритм визначення this (по кроках)

- 1. **Це стрілочна функція?** Так -> `this` те саме, що в зовнішній області видимості (де функція оголошена)
- 2. **Використано call/apply/bind?** Так -> `this` -- об'єкт, переданий їм першим аргументом
- 3. **Виклик як метод об'єкта** (`object.method()`)? Так -> `this` -- об'єкт зліва від крапки
- 4. **Суворий режим?** Так -> `this` -- `undefined` . Ні -> `this` -- `window`

Головна пастка: якщо витягнути метод з об'єкта в окрему змінну (`const fn = obj.method`) і викликати його окремо -- контекст **втрачається**, `this` вже не вказує на `obj` .

2. call, apply, bind

Метод	Синтаксис	Коли виконується	Аргументи функції
<code>call</code>	<code>fn.call(thisArg, a, b)</code>	одразу	перечислені через кому
<code>apply</code>	<code>fn.apply(thisArg, [a, b])</code>	одразу	
<code>bind</code>	<code>const bound = fn.bind(thisArg)</code>	пізніше -- повертає нову функцію	<code>bound(...)</code>

```
function greet(str) {
  console.log(`${str}, ${this.username}!`);
}
const mango = { username: "Mango" };

greet.call(mango, "Welcome"); // одразу, аргументи через кому
greet.apply(mango, ["Aloha"]); // одразу, аргументи масивом

const boundGreet = greet.bind(mango); // нова функція з "приклеєним" this
boundGreet("Hi"); // можна викликати пізніше
```

bind() рятує колбеки: якщо передати `customer.getFullName` як колбек напряму -- контекст загубиться. `customer.getFullName.bind(customer)` зберігає правильний `this` назавжди.

3. this у стрілочних функціях

	Звичайна функція	Стрілочна функція
<code>this</code> визначається	у момент виклику	у момент оголошення (з батьківської області)
<code>call/apply/bind</code>	змінюють <code>this</code>	не впливають на <code>this</code>
У суворому режимі	<code>this</code> -- <code>undefined</code>	ігнорує суворий режим, <code>this</code> -- зовнішній контекст

```
const hotel = {
  username: "Resort",
  showThis() {
    const foo = () => console.log(this); // стрілка -- запам'ятала контекст showThis
    foo();
  },
};
hotel.showThis(); // this -- об'єкт hotel (не window!)
```

4. Прототипи

```
const animal = { legs: 4 };
const dog = Object.create(animal); // dog отримує animal як прототип
dog.name = "Mango";

dog.name; // "Mango" -- власна властивість
dog.legs; // 4 -- не власна, знайдена в прототипі animal
```

Метод / оператор	Призначення
<code>Object.create(obj)</code>	створює новий об'єкт з <code>obj</code> як прототипом
<code>obj.hasOwnProperty(key)</code>	чи є властивість власною (не з прототипу)
<code>a.isPrototypeOf(b)</code>	чи є <code>a</code> прототипом для <code>b</code>
<code>Object.getPrototypeOf(obj)</code>	повертає прототип об'єкта напряду (сучасний спосіб замість читання <code>[[Prototype]]</code> у консолі)
<code>obj instanceof Class</code>	перевіряє, чи <code>Class.prototype</code> присутній у ланцюжку прототипів <code>obj</code> -- стандартний спосіб дізнатись "чи є obj екземпляром цього класу"

```
console.log(Object.getPrototypeOf(dog) === animal); // true
console.log(dog instanceof Object); // true -- будь-який об'єкт -- екземпляр Object
```

Ланцюжок прототипів: якщо властивість не знайдено в об'єкті, пошук триває в його прототипі, потім у прототипі прототипу і так далі, поки не дійде до `Object.prototype` (кінець ланцюжка) -- або поверне `undefined`, якщо ніде не знайдено.

```
for (const key in dog) { /* побачить і "name", і "legs" -- не розрізняє власне/невласне */ }

// Щоб отримати тільки власні -- Object.keys()/Object.values() або перевірка hasOwnProperty
Object.keys(dog); // ["name"] -- тільки власні
```

5. ООП: базові поняття

Термін	Значення
Клас	шаблон/креслення, що описує структуру та поведінку майбутніх об'єктів
Екземпляр (інстанція)	конкретний об'єкт, створений за класом, зі своїми значеннями властивостей
Інтерфейс класу	набір публічних властивостей і методів, доступних ззовні для роботи з екземпляром
Інкапсуляція	приховування деталей внутрішньої реалізації (приватні властивості/методи)

Процедурний підхід (окремі функції + глобальні змінні) добре працює для простих задач, але втрачає зв'язок між даними й логікою їх обробки зі зростанням складності -- звідси й потреба в ООП.

6. Класи: оголошення та конструктор

```
class User {
  constructor(params) {
    this.name = params.name; // this -- новостворений екземпляр
    this.email = params.email;
  }
}

const mango = new User({ name: "Mango", email: "mango@mail.com" });
// new -- створює об'єкт і автоматично викликає constructor
```

Методи класу

```
class User {
  constructor(params) { this.email = params.email; }

  getEmail() { return this.email; } // публічний метод
  changeEmail(newEmail) { this.email = newEmail; }
}
```

Методи класу зберігаються не на кожному екземплярі, а в `User.prototype` -- одна функція, спільна для всіх екземплярів. Саме тому екземпляри такі "легкі" в пам'яті.

7. Приватні властивості, геттери/сеттери, статика

```
class User {
  #email; // приватна властивість (обов'язково оголосити заздалегідь)

  constructor(params) { this.#email = params.email; }

  get email() { return this.#email; } // геттер -- виглядає як звичайна властивість
  set email(newEmail) { // сеттер -- дозволяє валідацію
    if (!newEmail.includes("@")) return;
    this.#email = newEmail;
  }

  static roles = { admin: "admin", basic: "basic" }; // статична властивість -- доступна тільки класу
  static isValidRole(role) { return Object.values(User.roles).includes(role); } // статичний метод
}

mango.email; // виклик геттера, без дужок
mango.email = "new@x.com"; // виклик сеттера
User.roles.admin; // статика -- через клас, не через екземпляр
```

Модифікатор	Синтаксис	Доступ
Приватна властивість/метод	<code>#name</code>	тільки всередині класу
Геттер/сеттер	<code>get x() {}</code> / <code>set x(v) {}</code>	як властивість, без дужок при виклику
Статична властивість/метод	<code>static x</code>	тільки через сам клас (<code>Class.x</code>), не через екземпляр

8. Наслідування: extends, super

```
class User {
  #email;
  constructor(email) { this.#email = email; }
  get email() { return this.#email; }
  greet() { return `Hello, I am a user`; }
}

class Admin extends User {
  constructor(params) {
    super(params.email); // ОБОВ'ЯЗКОВО -- виклик конструктора батька перед this
    this.access = params.access;
  }
  // Новий метод, якого немає в User
  blacklist(email) { /* ... */ }
}
```

Без виклику `super(...)` у конструкторі дочірнього класу -- помилка при спробі звернутись до `this . super(args)` має викликатись **першим рядком** конструктора.

Перевизначення методу (override) і super.method()

```
class Admin extends User {
  constructor(params) {
    super(params.email);
    this.access = params.access;
  }

  // Перевизначаємо метод greet() з батьківського класу
  greet() {
    const parentGreeting = super.greet(); // викликаємо батьківську версію методу
    return `${parentGreeting}, and also an admin with ${this.access} access`;
  }
}

const admin = new Admin({ email: "a@mail.com", access: "superuser" });
admin.greet(); // "Hello, I am a user, and also an admin with superuser access"
```

`super.method()` дозволяє викликати батьківську версію методу навіть після того, як дочірній клас її перевизначив -- корисно, коли треба **доповнити** поведінку батька, а не повністю її замінити.

9. Швидкий довідник

Концепція / синтаксис	Призначення
this	посилання на об'єкт, у контексті якого викликана функція
fn.call(obj, a, b)	викликати fn з this=obj, аргументи через кому
fn.apply(obj, [a, b])	те саме, але аргументи масивом
fn.bind(obj)	повертає нову функцію з назавжди прив'язаним this
Object.create(obj)	створити об'єкт з obj як прототипом
obj.hasOwnProperty(key)	чи властивість власна (не успадкована)
a.isPrototypeOf(b)	чи a є прототипом для b
Object.getPrototypeOf(obj)	дістати прототип об'єкта
obj instanceof Class	чи obj -- екземпляр Class (по ланцюжку прототипів)
class X { constructor() {} }	оголошення класу з конструктором
new X(...)	створити екземпляр класу, викликає constructor
#prop	приватна властивість/метод -- доступ лише всередині класу
get x() / set x(v)	геттер / сеттер -- доступ до властивості як до звичайної
static prop / static method()	належить класу, не екземпляру
class Child extends Parent	наслідування від батьківського класу
super(args)	виклик конструктора батька (обов'язково першим рядком)
super.method()	виклик батьківської версії перевизначеного методу